UNIVERSIDADE PAULISTA – UNIP

Projeto Integrado Multidisciplinar

Curso Superior de Tecnologia em

Análise e Desenvolvimento de Sistemas

BIANCA DE FÁTIMA PINA – RA: F366693

FERNANDA LEMES – RA: H70JCJ4

GUSTAVO APOLINÁRIO SOUZA SANTOS – RA: R222430

MONIQUE SANTONE DE MOURA – RA: R155009 / VITOR HUGO CINCERRE ANGELO – RA: H74GID8

DESENVOLVIMENTO E IMPLANTAÇÃO DE UMA SOLUÇÃO TECNOLÓGICA CORPORATIVA INTEGRADA PARA APOIAR A TRANSFORMAÇÃO DIGITAL DA ONG VESTINDO SONHOS

Plataforma Web em ASP.NET Core MVC, API RESTful, Aplicativo Móvel em Flutter, Banco de Dados MySQL e Infraestrutura Cloud/DevOps

ARARAQUARA
2026

BIANCA DE FÁTIMA PINA – RA: F366693

FERNANDA LEMES – RA: H70JCJ4

GUSTAVO APOLINÁRIO SOUZA SANTOS – RA: R222430

MONIQUE SANTONE DE MOURA – RA: R155009 / VITOR HUGO CINCERRE ANGELO – RA: H74GID8

DESENVOLVIMENTO E IMPLANTAÇÃO DE UMA SOLUÇÃO TECNOLÓGICA CORPORATIVA INTEGRADA PARA APOIAR A TRANSFORMAÇÃO DIGITAL DA ONG VESTINDO SONHOS

Plataforma Web em ASP.NET Core MVC, API RESTful, Aplicativo Móvel em Flutter, Banco de Dados MySQL e Infraestrutura Cloud/DevOps

Projeto Integrado Multidisciplinar em

Análise e Desenvolvimento de Sistemas

Projeto Integrado Multidisciplinar para obtenção do título de tecnólogo em Análise e Desenvolvimento de Sistemas, apresentado à Universidade Paulista – UNIP.

Orientador: Prof. Me. Salvador T. de Oliveira

ARARAQUARA
2026

RESUMO

A transformação digital no terceiro setor constitui um fator determinante para ampliar o alcance social, a transparência contábil e a eficiência operacional de organizações não governamentais. Este Projeto Integrado Multidisciplinar IV (PIM IV) apresenta o desenvolvimento e a implantação de uma solução tecnológica corporativa integrada para a ONG Vestindo Sonhos, entidade sem fins lucrativos dedicada à arrecadação, triagem e distribuição de vestuário, calçados, alimentos e recursos financeiros para famílias em situação de vulnerabilidade socioeconômica. Dando continuidade aos levantamentos arquiteturais concebidos no PIM III, o trabalho integra os conteúdos de Empreendedorismo em Tecnologia da Informação, Relações Étnico-Raciais e Afrodescendência, Desenvolvimento Web com .NET, Desenvolvimento Mobile, Programação Aplicada em .NET, Programação de Banco de Dados, Computação em Nuvem com DevOps e Gerenciamento Ágil de Projetos. A metodologia adotada combina revisão bibliográfica fundamentada na literatura de engenharia de software e administração de sistemas com o desenvolvimento experimental de um ecossistema tecnológico multiplataforma. No back-end e na camada web, foi construída uma aplicação em C# utilizando o framework ASP.NET Core (.NET 10) sob o padrão arquitetural Model-View-Controller (MVC), integrada a uma camada de serviços RESTful com autenticação híbrida baseada em Cookies (para o portal administrativo) e JSON Web Tokens (JWT Bearer) para clientes móveis, além de criptografia de senhas via algoritmo PBKDF2 e controle de acesso baseado em papéis (Administrador e Colaborador). No segmento móvel, desenvolveu-se um aplicativo nativo multiplataforma em Flutter (Dart) com interface Material Design 3, permitindo aos voluntários e colaboradores em campo consultar indicadores em tempo real, gerenciar o estoque de doações, cadastrar beneficiários e seus dependentes, registrar movimentações financeiras e acompanhar campanhas solidárias. A persistência de dados foi estruturada no Sistema Gerenciador de Banco de Dados MySQL 8.0, contemplando modelagem relacional normalizada até a Terceira Forma Normal (3FN), integridade referencial, Views analíticas, Stored Procedures para consolidação financeira e Triggers transacionais para atualização automatizada de saldos de inventário. Adicionalmente, definiu-se a arquitetura de implantação em nuvem com conteinerização Docker e pipeline de Integração e Entrega Contínua (CI/CD) via GitHub Actions, gerenciada por meio das práticas ágeis Scrum e Kanban. Os resultados demonstram que a solução elimina retrabalhos manuais, assegura rastreabilidade integral das doações e fortalece a promoção da cidadania, da inclusão digital e da diversidade étnico-racial.

Palavras-chave: Transformação Digital. Terceiro Setor. ASP.NET Core. Flutter. Banco de Dados Relacional.

ABSTRACT

Digital transformation in the non-profit sector is a decisive factor in expanding social reach, accounting transparency, and operational efficiency for non-governmental organizations. This Multidisciplinary Integrated Project IV (PIM IV) presents the development and deployment of an integrated corporate technological solution for the NGO Vestindo Sonhos, a non-profit organization dedicated to collecting, sorting, and distributing clothing, footwear, food, and financial resources to families experiencing socioeconomic vulnerability. Continuing the architectural studies conducted in PIM III, this work integrates knowledge from IT Entrepreneurship, Ethnic-Racial Relations and Afro-descendance, Web Development with .NET, Mobile Development, Applied .NET Programming, Database Programming, Cloud Computing with DevOps, and Agile Project Management. The methodology combines literature review in software engineering and information systems with the experimental development of a multiplatform technological ecosystem. On the back-end and web tier, an application was built in C# using the ASP.NET Core (.NET 10) framework following the Model-View-Controller (MVC) architectural pattern, integrated with a RESTful API layer featuring hybrid authentication based on Cookies (for the administrative web portal) and JSON Web Tokens (JWT Bearer) for mobile clients, alongside PBKDF2 password hashing and Role-Based Access Control (Administrator and Collaborator roles). For the mobile tier, a cross-platform native application was developed in Flutter (Dart) using Material Design 3 guidelines, enabling field volunteers and collaborators to view real-time indicators, manage donation inventory, register beneficiaries and their dependents, record financial transactions, and monitor solidarity campaigns. Data persistence was implemented in MySQL 8.0, featuring a relational model normalized to the Third Normal Form (3NF), referential integrity constraints, analytical Views, Stored Procedures for financial reporting, and transactional Triggers for automated inventory balance updates. Furthermore, a cloud deployment architecture was defined using Docker containers and a Continuous Integration and Continuous Delivery (CI/CD) pipeline via GitHub Actions, managed through Scrum and Kanban agile practices. The results demonstrate that the solution eliminates manual rework, ensures end-to-end traceability of donations, and strengthens citizenship, digital inclusion, and ethnic-racial diversity.

Keywords: Digital Transformation. Non-Profit Sector. ASP.NET Core. Flutter. Relational Database.

SUMÁRIO

INTRODUÇÃO 6
1 CARACTERIZAÇÃO DA ORGANIZAÇÃO 8
1.1 Segmento de atuação e papel social no terceiro setor 8
1.2 Produtos, serviços e ações sociais prestadas 8
1.3 Estrutura organizacional e perfis de acesso 9
1.4 Mapeamento dos principais processos de negócio 10
1.5 Problemas identificados no cenário pré-implantação 10
1.6 Oportunidades de melhoria por meio da tecnologia 11
2 PLANEJAMENTO DA SOLUÇÃO TECNOLÓGICA (EMPREENDEDORISMO EM TI) 11
2.1 Problema a ser resolvido e justificativa empreendedora 11
2.2 Objetivos da solução tecnológica 12
2.3 Público-alvo e partes interessadas (stakeholders) 12
2.4 Proposta de valor e modelo de negócio social (Canvas) 12
2.5 Benefícios esperados e análise de viabilidade 13
2.6 Diferenciais competitivos e inovação social 14
3 RESPONSABILIDADE SOCIAL, DIVERSIDADE E RELAÇÕES ÉTNICO-RACIAIS 14
3.1 Combate à discriminação e valorização da afrodescendência 14
3.2 Valorização da diversidade no acolhimento de famílias 15
3.3 Recursos tecnológicos inclusivos e acessibilidade digital 15
3.4 Utilização da tecnologia para promoção da cidadania 16
4 DESENVOLVIMENTO DA SOLUÇÃO WEB (DESENVOLVIMENTO WEB COM .NET) 17
4.1 Arquitetura da aplicação web ASP.NET Core MVC (.NET 10) 17
4.2 Construção das APIs RESTful e contratos de integração 17
4.3 Autenticação híbrida (Cookies e JWT) e segurança da informação 19
4.4 Estrutura administrativa e integração entre módulos web 20
5 DESENVOLVIMENTO DA SOLUÇÃO MOBILE (DESENVOLVIMENTO MOBILE) 20
5.1 Arquitetura do aplicativo móvel em Flutter e Material Design 3 21
5.2 Telas principais e fluxo de navegação por perfil 21
5.3 Autenticação JWT, sincronização de dados e consumo das APIs REST 22
6 ARQUITETURA DE SOFTWARE (PROGRAMAÇÃO APLICADA EM .NET) 23
6.1 Orientação a objetos e princípios SOLID aplicados 23
6.2 Organização em camadas, modularização, ViewModels e DTOs 24
7 PROJETO E PROGRAMAÇÃO DO BANCO DE DADOS 25
7.1 Modelo conceitual (MER) e modelo lógico normalizado (3FN) 25
7.2 Modelo físico e dicionário das tabelas do MySQL 8.0 26
7.3 Stored Procedures, Triggers e Views implementadas 28
7.4 Script completo do banco de dados (MySQL 8.0) 28
8 INFRAESTRUTURA EM NUVEM E DEVOPS (CLOUD COMPUTING E DEVOPS) 31
8.1 Arquitetura em nuvem, serviços utilizados e containers Docker 32
8.2 Pipeline CI/CD com GitHub Actions, monitoramento, escalabilidade e segurança 32
9 GERENCIAMENTO ÁGIL DO PROJETO 33
9.1 Framework Scrum/Kanban, Product Backlog e Sprint Backlog 33
9.2 Cronograma das Sprints, Quadro Kanban e definição das entregas 34
CONCLUSÃO 36
REFERÊNCIAS 37

INTRODUÇÃO

A evolução acelerada das Tecnologias da Informação e Comunicação (TICs) remodelou profundamente a dinâmica produtiva, gerencial e administrativa das organizações contemporâneas. Embora o fenômeno da transformação digital tenha sido inicialmente impulsionado por corporações do setor privado em busca de eficiência operacional e vantagem competitiva, sua adoção pelas instituições do terceiro setor tornou-se indispensável para assegurar governança transparente, agilidade no atendimento social e ampliação do alcance comunitário. Organizações Não Governamentais (ONGs) operam cotidianamente em cenários de alta complexidade, lidando com recursos materiais e financeiros limitados, demandas sociais crescentes e a necessidade estrita de prestação de contas perante doadores, voluntários e órgãos reguladores.

Nesse contexto insere-se a ONG Vestindo Sonhos, instituição filantrópica objeto deste estudo, cuja missão institucional consiste em arrecadar, triar, estocar e distribuir vestuário, calçados, agasalhos, alimentos não perecíveis e auxílios emergenciais para famílias em situação de vulnerabilidade socioeconômica, além de organizar bazares beneficentes e campanhas sazonais de solidariedade. Conforme diagnosticado na etapa precedente deste projeto acadêmico (PIM III), a operação da entidade apoiava-se anteriormente em controles manuais, fichas em papel e planilhas eletrônicas descentralizadas, o que resultava em divergências recorrentes no saldo de inventário, morosidade na triagem de beneficiários e seus dependentes, dificuldades na coordenação de voluntários em eventos externos e ausência de consolidação financeira em tempo real.

Diante dessa problemática, o objetivo geral deste Projeto Integrado Multidisciplinar IV (PIM IV) consiste em desenvolver e implantar uma solução tecnológica corporativa completa, modular e integrada para apoiar a transformação digital da ONG Vestindo Sonhos. A solução materializa o planejamento arquitetural definido no PIM III por meio da implementação prática de quatro pilares tecnológicos: (i) uma aplicação Web administrativa em C# ASP.NET Core MVC (.NET 10); (ii) uma camada de APIs RESTful com autenticação híbrida; (iii) um aplicativo móvel nativo multiplataforma em Flutter (Dart) voltado à atuação de voluntários e colaboradores em campo; e (iv) um banco de dados relacional corporativo em MySQL 8.0 com automações transacionais internas, sustentados por

infraestrutura em nuvem, pipeline DevOps de Integração Contínua (CI/CD) e gerenciamento ágil de projetos.

Do ponto de vista metodológico, a pesquisa classifica-se, segundo a taxonomia de Gil (2019) e Lakatos e Marconi (2021), como uma pesquisa aplicada de abordagem quali-quantitativa, articulando pesquisa bibliográfica, estudo de caso organizacional e desenvolvimento experimental de software. Conforme preconizam Severino (2017) e Cervo, Bervian e Silva (2007), o rigor metodológico exige que a construção prática do artefato tecnológico seja sustentada por fundamentação teórica consolidada. Assim, a arquitetura de software e a orientação a objetos baseiam-se em Pressman e Maxim (2021), Sommerville (2019), Fowler (2018) e Martin (2020); o projeto de banco de dados apoia-se em Elmasri e Navathe (2019) e Date (2004); o planejamento empreendedor fundamenta-se em Dornelas (2021) e Osterwalder e Pigneur (2020); a governança DevOps segue Kim et al. (2021); e a gestão ágil adota o Guia do Scrum de Schwaber e Sutherland (2020).

Para atender integralmente às nove etapas previstas no Manual do PIM IV do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade Paulista (UNIP), o desenvolvimento deste trabalho está organizado em nove capítulos sequenciais. O Capítulo 1 realiza a caracterização da ONG Vestindo Sonhos. O Capítulo 2 apresenta o planejamento empreendedor da solução tecnológica e sua análise de viabilidade. O Capítulo 3 discute as práticas de responsabilidade social, diversidade étnico-racial e acessibilidade digital incorporadas à plataforma. O Capítulo 4 detalha o desenvolvimento da solução Web e das APIs REST em .NET 10. O Capítulo 5 expõe a construção do aplicativo móvel em Flutter. O Capítulo 6 analisa a arquitetura de software orientada a objetos e sua divisão em camadas. O Capítulo 7 documenta o projeto lógico e físico do banco de dados MySQL 8.0, incluindo Stored Procedures, Triggers e o Script SQL completo. O Capítulo 8 descreve o plano de infraestrutura em nuvem e a esteira DevOps CI/CD. Por fim, o Capítulo 9 apresenta o gerenciamento ágil do projeto com Scrum e Kanban, seguido da Conclusão e das Referências bibliográficas.

1 CARACTERIZAÇÃO DA ORGANIZAÇÃO

A engenharia de requisitos e a modelagem de sistemas corporativos exigem uma imersão prévia na realidade operacional da organização-alvo. Segundo Sommerville (2019), sistemas de informação bem-sucedidos são aqueles que compreendem o ecossistema sociotécnico no qual serão inseridos, respeitando os processos organizacionais e eliminando gargalos produtivos. Neste primeiro capítulo, apresenta-se o diagnóstico completo da ONG Vestindo Sonhos.

1.1 Segmento de atuação e papel social no terceiro setor

A ONG Vestindo Sonhos atua no Terceiro Setor brasileiro, especificamente no segmento de assistência social comunitária, segurança alimentar e promoção da dignidade humana por meio da doação organizada de vestuário e insumos básicos. Constituída juridicamente como associação civil de direito privado sem fins lucrativos, a instituição concentra suas atividades no atendimento a famílias em situação de vulnerabilidade socioeconômica na região metropolitana de São Paulo.

O terceiro setor exerce uma função vital na redução das assimetrias sociais brasileiras, atuando onde o poder público muitas vezes não consegue alcançar com a velocidade necessária em situações de emergência climática (como ondas de frio intenso ou enchentes) e desemprego estrutural. A ONG Vestindo Sonhos articula uma rede solidária composta por doadores individuais, comércios parceiros, voluntários comunitários e assistentes sociais para transformar doações físicas e financeiras em amparo direto às famílias cadastradas.

1.2 Produtos, serviços e ações sociais prestadas

Embora não comercialize produtos com fins lucrativos, a ONG Vestindo Sonhos opera uma cadeia logística e assistencial completa, estruturada em quatro serviços essenciais prestados gratuitamente à comunidade:

- Arrecadação, triagem e distribuição de vestuário, calçados e alimentos: recebimento contínuo de peças de roupa (camisetas, calças, casacos, meias), calçados e cestas de alimentos, que passam por inspeção física, higienização, classificação por categoria de ativo (`TiposAtivo`), tamanho (`P`, `M`, `G`, `GG`, infantil ou numeração) e cor, sendo armazenados no estoque central para entrega controlada;

- Cadastro e acompanhamento sociofamiliar (Beneficiários e Dependentes): acolhimento humanizado de responsáveis familiares (`Beneficiario`), registrando endereço, telefone, CPF, situação cadastral (`Ok`, `Aguardando Doação` ou `Bloqueado`) e mapeamento individualizado de todos os seus dependentes (`Dependente`), incluindo idade, sexo e vínculo de parentesco;
- Organização de campanhas sazonais e bazares comunitários (`Eventos`): planejamento e execução de ações coletivas, como a Campanha do Agasalho de Inverno e o Bazar Beneficente de Outono, realizados na sede da ONG ou em ginásios municipais com alocação estruturada de voluntários;
- Captação e gestão transparente de recursos financeiros (`Financeiro` e `Carteiras`): administração de doações pecuniárias recebidas via PIX, transferências bancárias ou contribuições presenciais em espécie, destinadas à compra suplementar de cobertores, alimentos e manutenção operacional da sede.

1.3 Estrutura organizacional e perfis de acesso

A governança interna da ONG Vestindo Sonhos é enxuta e horizontalizada, organizada em três esferas de atuação que foram diretamente traduzidas no modelo de Controle de Acesso Baseado em Papéis (RBAC — Role-Based Access Control) da solução tecnológica, conforme detalhado na Tabela 1.

Tabela 1 – Estrutura organizacional da ONG Vestindo Sonhos e mapeamento de perfis no sistema

Nível Organizacional	Atribuições na Instituição	Perfil no Sistema (Role)	Escopo de Permissões Web e Mobile
Diretoria Executiva e Conselho Fiscal	Gestão estratégica, administração de contas/carteiras bancárias, auditoria de receitas e despesas, gestão de cargos e cadastro de usuários.	Administrador	Acesso irrestrito: Dashboard, Financeiro, Carteiras, Categorias, Inventário, Produtos, Beneficiários, Dependentes, Eventos, Voluntários, Usuários e Cargos.
Corpo de Colaboradores e Assistentes Sociais	Triagem presencial de famílias, cadastro de dependentes, movimentação de entrada/saída de estoque e organização de eventos e voluntários.	Colaborador	Acesso operacional completo na Web e no Mobile: Home/Dashboard, Inventário, Produtos, Beneficiários, Dependentes, Eventos e Voluntários (sem acesso ao módulo Financeiro).

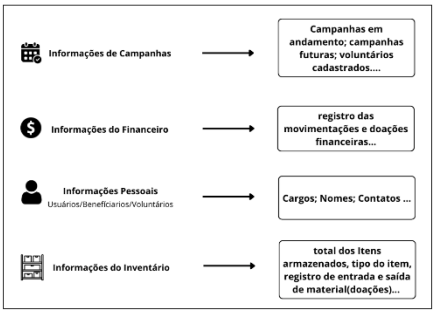
Nível Organizacional	Atribuições na Instituição	Perfil no Sistema (Role)	Escopo de Permissões Web e Mobile
Rede de Voluntários Comunitários	Atuação presencial em mutirões de coleta, separação de roupas e apoio logístico nos eventos beneficentes.	Voluntário Cadastrado	Registrado na base de Voluntários e vinculado nominalmente aos eventos na tabela associativa VoluntarioAtribuido.

Fonte: Elaborada pelos autores (2026).

1.4 Mapeamento dos principais processos de negócio

O funcionamento cotidiano da organização apoia-se em quatro macroprocessos integrados: (1) Processo de Gestão de Estoque de Doações, que compreende o cadastro prévio de categorias de ativos e produtos, o lançamento de transações de entrada (quando um lote de doações chega à sede) e o lançamento de transações de saída (quando itens são entregues a um beneficiário específico); (2) Processo de Acolhimento de Beneficiários, no qual o colaborador registra o responsável familiar, anota em formato de hashtags (`#Calças`, `#Comida`, `#Roupas`, `#Meias`) as necessidades urgentes da família e cadastra cada filho ou dependente com sua respectiva idade; (3) Processo de Gestão de Eventos e Voluntariado, que agenda campanhas solidárias, designa um colaborador responsável e atribui a equipe de voluntários; e (4) Processo de Gestão Financeira, que escritura cada entrada de doação ou saída de despesa associando-a a uma carteira e categoria contábil.

Figura 1 – Segmentação das informações essenciais da ONG



Fonte: Elaborado pelos autores (2026).

1.5 Problemas identificados no cenário pré-implantação

Antes do desenvolvimento da solução tecnológica objeto deste trabalho, a operação da ONG sofria com cinco problemas críticos diagnosticados no estudo preliminar:

- Falta de integridade no controle de estoque: o uso de planilhas isoladas permitia erros de digitação, duplicidade de produtos e baixa de quantidades inexistentes, gerando constrangimento quando um item constava como disponível na planilha, mas já havia acabado fisicamente no galpão;
- Impossibilidade de operação móvel em eventos externos: durante campanhas realizadas em ginásios e praças públicas, os colaboradores utilizavam pranchetas de papel para anotar entregas e novos cadastros, gerando retrabalho de digitação posterior e perda de fichas;
- Ausência de visão analítica sobre as carências da comunidade: a coordenação não conseguia identificar rapidamente quais itens estavam sendo mais demandados pelas famílias para direcionar as campanhas de arrecadação;
- Fragilidade no controle de permissões e segurança: não havia separação sistêmica entre o acesso operacional dos voluntários/colaboradores e os registros financeiros das contas bancárias da ONG;
- Dificuldade no acompanhamento dos dependentes familiares: falhas no registro da idade e do sexo das crianças dependentes prejudicavam a separação adequada de roupas e calçados infantis.

1.6 Oportunidades de melhoria por meio da tecnologia

A transformação digital implementada neste PIM IV converte cada um desses gargalos em eficiência mensurável: a centralização em banco de dados MySQL com Triggers automáticas garante estoque fidedigno em tempo real; o aplicativo móvel em Flutter conectado via Wi-Fi/rede à API REST elimina o uso de papel nos eventos externos; o painel de Dashboard processa automaticamente as tags de necessidade das famílias para orientar as campanhas; e o controle de acesso por cargos (`Administrador` e `Colaborador`) protege os dados financeiros sem restringir a produtividade da equipe operacional.

2 PLANEJAMENTO DA SOLUÇÃO TECNOLÓGICA (EMPREENDEDORISMO EM TI)

Na disciplina de Empreendedorismo em Tecnologia da Informação, compreende-se que o desenvolvimento de um software corporativo deve ser orientado pela geração de valor tangível, sustentabilidade operacional e resolução efetiva de problemas reais (DORNELAS, 2021). No terceiro setor, o empreendedorismo tecnológico assume caráter de inovação social, buscando maximizar o benefício coletivo com o menor custo de infraestrutura possível.

2.1 Problema a ser resolvido e justificativa empreendedora

As organizações filantrópicas enfrentam um dilema recorrente: sistemas de gestão empresarial (ERPs) tradicionais disponíveis no mercado possuem mensalidades elevadas e são voltados ao comércio varejista ou indústria (faturamento, emissão de notas fiscais de venda, cálculo de impostos), não atendendo às especificidades de uma ONG — como controle de beneficiários por situação social, gestão de dependentes, alocação de voluntários em campanhas e rastreamento de doações recebidas e entregues. O projeto Vestindo Sonhos preenche essa lacuna de mercado ao entregar um ERP Social completo, gratuito e especializado na rotina de entidades assistenciais.

2.2 Objetivos da solução tecnológica

A solução tecnológica foi planejada com os seguintes objetivos estratégicos e operacionais: (1) unificar em uma única base de dados relacional todas as informações de beneficiários, dependentes, estoque de produtos, transações

financeiras, carteiras, eventos e voluntários; (2) disponibilizar acesso multiplataforma simultâneo via navegador Web (para gestão administrativa na sede) e aplicativo móvel Android/iOS (para atuação ágil em campo); (3) automatizar o cálculo de saldos de estoque e de carteiras financeiras, impedindo inconsistências contábeis; e (4) fornecer relatórios visuais e exportação de dados em formato compatível com Excel (CSV UTF-8) para prestação de contas anual.

2.3 Público-alvo e partes interessadas (stakeholders)

O público-alvo direto da solução compreende os gestores administrativos, coordenadores de projetos, assistentes sociais, colaboradores e voluntários da ONG Vestindo Sonhos, podendo ser facilmente replicado para outras associações beneficentes, igrejas, centros comunitários e fundações de assistência social. Como público-alvo indireto figuram as centenas de famílias beneficiárias atendidas pela instituição — que passam a contar com um atendimento digno, rápido e organizado — e os doadores pessoas físicas e jurídicas, que ganham transparência sobre a destinação de suas contribuições.

2.4 Proposta de valor e modelo de negócio social (Canvas)

A proposta de valor central da plataforma consiste em oferecer governança corporativa de nível empresarial para uma instituição filantrópica, unindo simplicidade de uso (incluindo inicialização portátil em um clique) e robustez tecnológica (.NET 10, Flutter e MySQL 8.0). A estruturação completa do modelo de negócio segundo a metodologia Business Model Canvas (OSTERWALDER; PIGNEUR, 2020) é apresentada na Tabela 2.

Tabela 2 – Business Model Canvas Social da solução tecnológica ONG Vestindo Sonhos

Dimensão do Canvas	Detalhamento Estratégico no Projeto Vestindo Sonhos
Proposta de Valor	Plataforma integrada Web e Mobile para gestão transparente de doações, controle automatizado de estoque, acolhimento ágil de famílias vulneráveis e gestão de eventos e voluntários.
Segmentos de Clientes / Usuários	Gestores e colaboradores da ONG Vestindo Sonhos (usuários diretos); famílias em vulnerabilidade social e doadores mantenedores (beneficiados indiretos).
Canais de Acesso e Entrega	Portal Web MVC responsivo, Aplicativo Móvel Android (VestindoSonhos_Mobile.apk), Web App Mobile embarcado (/mobile) e pacote portátil Windows (VestindoSonhos_Portavel).

Dimensão do Canvas	Detalhamento Estratégico no Projeto Vestindo Sonhos
Relacionamento com Usuários	Atendimento humanizado apoiado por histórico familiar completo, segurança de acesso com troca obrigatória de senha e transparência contábil.
Fontes de Sustentabilidade	Arrecadação de doações financeiras (auditadas por carteira), bazares comunitários, parcerias privadas de responsabilidade social (ESG) e editais públicos.
Recursos Principais	Aplicação ASP.NET Core 10, APIs RESTful, aplicativo Flutter, banco de dados MySQL 8.0 e pipeline automatizado no GitHub Actions.
Atividades-Chave	Recebimento e triagem de doações, controle de inventário, cadastro de beneficiários e dependentes, realização de campanhas e prestação de contas.
Parcerias Principais	Empresas doadoras de vestuário e alimentos, voluntários comunitários, lideranças locais e instituições acadêmicas (UNIP).
Estrutura de Custos	Custo zero de licenciamento de software (stack 100% open-source/free tier), exigindo apenas hospedagem enxuta em nuvem ou execução em servidor local da sede.

Fonte: Elaborada pelos autores (2026).

2.5 Benefícios esperados e análise de viabilidade

A análise de viabilidade do projeto confirma sua plena exequibilidade nas esferas técnica, operacional e econômica. Na esfera técnica, a combinação de ASP.NET Core (.NET 10) com Flutter e MySQL 8.0 representa o estado da arte em desempenho e segurança. Na esfera operacional, a solução elimina a necessidade de conhecimento técnico avançado por parte da ONG: além do funcionamento em nuvem, foi desenvolvido um empacotador automatizado (`gerar-executavel-portatil.bat`) que gera um executável autocontido (`VestindoSonhos.exe`) com todo o runtime .NET embutido, permitindo iniciar o sistema com um simples duplo clique (`1-CLIQUE_INICIAR_SISTEMA.bat`). Na esfera econômica, a inexistência de taxas de licença proporciona economia imediata e eleva a capacidade de captação de recursos da ONG.

2.6 Diferenciais competitivos e inovação social

Como diferenciais competitivos destacam-se: (a) a extração algorítmica de hashtags de necessidades sociais diretamente do cadastro de beneficiários para alimentar o Dashboard executivo e mobile; (b) a flexibilidade de conexão em campo do aplicativo Flutter, que conta com um diálogo dedicado (`ServerConfigDialog`) para apontar o aplicativo para o IP local do servidor durante mutirões sem depender

de internet externa; e (c) a sincronia em tempo real entre todas as operações realizadas no celular e no computador administrativo.

3 RESPONSABILIDADE SOCIAL, DIVERSIDADE E RELAÇÕES ÉTNICO-RACIAIS

O desenvolvimento de sistemas de informação voltados ao interesse público e ao terceiro setor exige uma reflexão ética profunda acerca das desigualdades históricas da sociedade brasileira. Em conformidade com as diretrizes curriculares da Lei nº 10.639/2003, da Lei nº 11.645/2008 e do Estatuto da Igualdade Racial — Lei nº 12.288/2010 (BRASIL, 2010), a solução tecnológica da ONG Vestindo Sonhos incorpora mecanismos concretos de promoção da equidade étnico-racial, valorização da diversidade, acessibilidade digital e fortalecimento da cidadania.

3.1 Combate à discriminação e valorização da afrodescendência

A formação socioeconômica brasileira legou um quadro de desigualdade racial estrutural no qual a população negra (pretos e pardos, segundo a classificação oficial do IBGE) e as comunidades periféricas enfrentam os maiores índices de vulnerabilidade material e insegurança alimentar. Reconhecer essa realidade é fundamental para que uma instituição assistencial atue como agente de transformação e justiça social, e não de mera reprodução de assimetrias.

Na plataforma Vestindo Sonhos, o combate a qualquer forma de discriminação étnico-racial, religiosa ou de gênero manifesta-se na padronização transparente e impessoal do fluxo de acolhimento. A classificação da situação do beneficiário (`Ok`, `Aguardando Doação`, `Bloqueado`) e o registro histórico de todas as entregas realizadas em `TransacaoInventario` garantem que a distribuição de roupas, calçados e alimentos ocorra com base na real necessidade familiar documentada e na ordem de acolhimento, coibindo favoritismos ou tratamentos discriminatórios. Ademais, o módulo de Eventos viabiliza a organização de ações afirmativas, feiras de empreendedorismo negro e campanhas culturais que celebram a história e a cultura afro-brasileira nas comunidades atendidas.

3.2 Valorização da diversidade no acolhimento de famílias

A concepção do módulo de Beneficiários e Dependentes rejeitou modelos cadastrais engessados que pressupõem apenas um formato único de família

tradicional. No Brasil contemporâneo, grande parte dos núcleos familiares vulneráveis é composta por mulheres chefes de família (especialmente mães solo negras), avós que criam netos ou famílias extensas solidárias. Por essa razão, a entidade `Dependente` foi modelada para acolher qualquer vínculo parental ou afetivo no atributo `Parentesco` (filho, filha, neto, sobrinho, enteado, irmão, cônjuge, agregado) e disponibiliza no atributo `Sexo` as opções `M` (Masculino), `F` (Feminino) e `n` (Não informado / Não binário / Outros), assegurando respeito integral à identidade de gênero e à diversidade familiar.

3.3 Recursos tecnológicos inclusivos e acessibilidade digital

Para garantir que voluntários e colaboradores de todas as faixas etárias, graus de escolaridade ou limitações visuais possam operar o sistema com autonomia, foram implementadas práticas de acessibilidade digital baseadas nas recomendações WCAG 2.1 (Web Content Accessibility Guidelines) e no Modelo de Acessibilidade em Governo Eletrônico (eMAG):

- Contraste visual elevado e semântica multimodal: todas as telas Web e Mobile utilizam tipografia sem serifa de alta legibilidade, botões com área de toque ampliada (mínimo de 48x48dp no mobile) e combinação de ícones ilustrativos com rótulos textuais claros, auxiliando usuários com daltonismo ou baixa acuidade visual;
- Responsividade elástica a fontes ampliadas: no aplicativo móvel (`home_screen.dart` e `financeiro_screen.dart`), os cartões de métricas utilizam containers flexíveis (`Row` + `Expanded` + `Flexible`) que se expandem verticalmente sem truncar informações quando o usuário ativa o recurso de Fonte Grande do sistema operacional Android;
- Prevenção de erros e assistência de digitação: os formulários Web e Mobile aplicam filtros automáticos que aceitam apenas dígitos em campos de CPF e telefone, exibem mensagens de validação em língua portuguesa clara e orientam o usuário passo a passo.

3.4 Utilização da tecnologia para promoção da cidadania

A tecnologia atua como instrumento de cidadania quando devolve visibilidade e dignidade às pessoas historicamente marginalizadas. Ao manter um prontuário social organizado de cada família e de seus dependentes — sabendo

exatamente quantas crianças existem em cada lar, suas idades e quais itens urgentes (`#Comida`, `#Calças`, `#Meias`, `#Roupas`) estão pendentes —, a ONG Vestindo Sonhos promove inclusão social qualitativa. Simultaneamente, o rigor na proteção dos dados pessoais dos beneficiários mediante autenticação restrita cumpre os princípios de finalidade, necessidade e segurança da Lei Geral de Proteção de Dados Pessoais — Lei nº 13.709/2018 (BRASIL, 2018).

Figura 2 – Recurso de acessibilidade em Libras (plugin VLibras) ativo no sistema



Fonte: Elaborado pelos autores (2026).

4 DESENVOLVIMENTO DA SOLUÇÃO WEB (DESENVOLVIMENTO WEB COM .NET)

O desenvolvimento da solução Web e da camada de serviços corporativos foi conduzido utilizando a plataforma Microsoft .NET 10 e a linguagem C#, adotando o framework ASP.NET Core MVC integrado ao Web API. Essa escolha arquitetural permite hospedar em um único processo de servidor de alta performance (Kestrel) tanto a interface administrativa rica renderizada em HTML5/Razor quanto os endpoints RESTful consumidos pelo aplicativo móvel.

4.1 Arquitetura da aplicação web ASP.NET Core MVC (.NET 10)

O projeto `VestindoSonhos` estrutura-se sob o padrão Model-View-Controller (MVC), promovendo clara separação entre a camada de apresentação (`Views/`), o controle de fluxo de requisições (`Controllers/`), os modelos de domínio (`Models/`) e os objetos de transferência de visualização (`ViewModels/`). No arquivo de inicialização `Program.cs`, o pipeline de execução é configurado com suporte a controladores com Views (`AddControllersWithViews`), provedor de banco de dados `MySql.EntityFrameworkCore` apontando para a base `VESTINDOSONHOSDB`, localização nativa em português do Brasil (`pt-BR`) e arquivos estáticos otimizados em `wwwroot/`.

Para garantir resiliência operacional em qualquer máquina onde o sistema seja implantado, implementou-se também em `Program.cs` uma rotina de auto-migração na inicialização (`Startup Schema Verification`): ao subir a aplicação, o sistema abre uma conexão com o MySQL, verifica se todas as colunas recentes (como `idade`, `observacao` e `ano_nascimento` na tabela `Dependente`) existem no banco local e, caso alguma esteja ausente em uma base antiga, executa automaticamente os comandos `ALTER TABLE` necessários sem intervenção humana.

4.2 Construção das APIs RESTful e contratos de integração

Para prover interoperabilidade com o aplicativo móvel Flutter e futuros sistemas parceiros, foram desenvolvidos sete controladores RESTful no diretório `VestindoSonhos/Controllers/Api/`, decorados com o atributo `[ApiController]` e roteamento semântico sob o prefixo `/api/`. As respostas utilizam objetos de transferência de dados (`Models/DTOs/`) serializados em formato JSON padrão CamelCase. A Tabela 3 apresenta o detalhamento técnico de todas as rotas da API REST construídas.

Tabela 3 – Especificação técnica das APIs RESTful implementadas no back-end ASP.NET Core (.NET 10)

Controlador REST	Verbo e Endpoint HTTP	Política de Acesso	Operação Realizada e Retorno JSON
AuthApiController	POST /api/auth/login	[AllowAnonymous]	Autentica usuário por e-mail e senha (com migração automática de hash PBKDF2), localiza vínculo de voluntário e retorna LoginResponseDto com Token JWT Bearer.
AuthApiController	POST /api/auth/trocar-senha-obrigatoria	[AllowAnonymous]	Permite ao usuário redefinir sua senha temporária inicial (123456) diretamente pelo aplicativo móvel, atualizando ExigirTrocaSenha = false.
DashboardApiController	GET /api/dashboard	Roles: Administrador, Colaborador	Consolida métricas em tempo real (projetos ativos, voluntários, estoque, beneficiários, saldos, ranking de tags

Controlador REST	Verbo e Endpoint HTTP	Política de Acesso	Operação Realizada e Retorno JSON
			#Necessidades e próximos 3 eventos).
BeneficiariosApiController	GET, POST, PUT, DELETE /api/beneficiarios	Roles: Administrador, Colaborador	Gerencia o ciclo de vida completo (CRUD) dos beneficiários responsáveis, retornando a lista aninhada de seus dependentes.
BeneficiariosApiController	POST, PUT, DELETE /api/beneficiarios/.../dependentes	Roles: Administrador, Colaborador	Adiciona, edita e remove dependentes vinculados a um beneficiário responsável (com validação de idade, sexo e parentesco).
InventarioApiController	GET /api/inventario e GET /api/inventario/opcoes	Roles: Administrador, Colaborador	Lista todos os produtos com suas quantidades atuais na tabela Inventario e fornece opções de TiposAtivo e Beneficiários.
InventarioApiController	POST, PUT, DELETE /api/inventario/produtos	Roles: Administrador, Colaborador	Cadastra novos produtos, edita atributos (nome, tamanho, cor, descrição, categoria) e realiza exclusão segura em cascata.
InventarioApiController	POST /api/inventario/movimentar	Roles: Administrador, Colaborador	Lança entradas de doações ou saídas para beneficiários em Transacao e TransacaoInventario, validando disponibilidade física.
InventarioApiController	POST, DELETE /api/inventario/tipos-ativo	Roles: Administrador, Colaborador	Permite criar e remover categorias de produtos (Roupas, Calçados, Alimentos, etc.) a partir do aplicativo.
FinanceiroApiController	GET /api/financeiro e POST /api/financeiro/transacao	Role: Administrador	Retorna resumo de carteiras, total de entradas/saídas e registra transações financeiras atualizando atomicamente o saldo da carteira.
EventosApiController	GET, POST, PUT, DELETE /api/eventos e /api/voluntarios	Roles: Administrador, Colaborador	Gerencia campanhas solidárias, cadastro de voluntários e

Controlador REST	Verbo e Endpoint HTTP	Política de Acesso	Operação Realizada e Retorno JSON
			vinculação/desvinculação em VoluntarioAtribuido.

Fonte: Elaborada pelos autores (2026).

4.3 Autenticação híbrida (Cookies e JWT) e segurança da informação

A arquitetura de segurança do sistema implementa um mecanismo de Autenticação Híbrida (`SmartAuthScheme`) configurado em `Program.cs`. Quando um usuário acessa o portal Web pelo navegador, o controlador `Login.cs` valida suas credenciais e emite um Cookie HTTP criptografado (`CookieAuthenticationDefaults.AuthenticationScheme`) contendo as *Claims* de identificação (`Name`, `Nome`, `UserId` e `Role`). Por outro lado, quando o aplicativo móvel consome qualquer rota iniciada por `/api`, o seletor encaminha a verificação para o middleware `JwtBearer`, que valida a assinatura criptográfica HMAC-SHA256 do token JWT enviado no cabeçalho `Authorization: Bearer`.

Além da proteção contra falsificação de requisições entre sites (`[ValidateAntiForgeryToken]` em todos os métodos `POST` dos controladores MVC), o sistema protege as senhas armazenadas utilizando a classe `PasswordHasher<Usuario>` (PBKDF2) e impõe a política de troca obrigatória de senha no primeiro login por meio do filtro global `ExigirTrocaSenhaFilter`.

4.4 Estrutura administrativa e integração entre módulos web

A interface administrativa Web foi construída com HTML5, CSS3 modularizado (`wwwroot/css/site.css`, `tabela.css`, `formularios.css`, `dashboard.css`, `financeiro.css`) e ícones vetoriais *Google Material Symbols*. Todos os módulos operam de forma integrada sobre o mesmo contexto transacional:

- Integração Beneficiários <-> Dependentes <-> Inventário: ao visualizar os detalhes de um beneficiário (`Views/Beneficiarios/Details.cshtml`), o colaborador visualiza e gerencia na mesma tela todos os seus dependentes e, ao registrar uma saída de doação no módulo de Inventário (`Views/Inventario/Criar.cshtml`), seleciona diretamente o beneficiário contemplado;

- Integração Inventário <-> Produtos <-> Tipos de Ativo: na tela principal do Inventário (`Views/Inventario/Index.cshtml`) e nos formulários de movimentação, botões dedicados permitem navegar diretamente para a listagem de Produtos (`Controllers/Produtos.cs`) ou cadastrar um Novo Produto sem interromper o fluxo operacional;
- Integração Financeiro <-> Carteiras <-> Categorias: toda transação lançada em `Controllers/Financeiro.cs` atualiza instantaneamente o atributo `Saldo` da respectiva `Carteira`, enquanto o controlador `Carteiras.cs` protege o saldo acumulado durante edições cadastrais e impede a exclusão de contas que possuam saldo diferente de zero (`Erro/ContaNaoVazia`);
- Integração Eventos <-> Voluntários <-> Usuários: cada evento criado em `Controllers/Eventos.cs` é vinculado a um usuário responsável e permite atribuir múltiplos voluntários cadastrados em `Controllers/Voluntarios.cs`.

Figura 3 – Controlador C# com autorização de perfil e proteção AntiForgeryToken



```
// GET: Carteiras/Criar
[Authorize(Roles = "Administrador")]
0 referências
public IActionResult Criar()
{
    return View();
}

// POST: Carteiras/Criar
// To protect from overposting attacks, enable the specific properties you want to bind to.
// For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
[Authorize(Roles = "Administrador")]
[HttpPost]
[ValidateAntiForgeryToken]
0 referências
public async Task<IActionResult> Criar([Bind("Id,Nome")] Carteira carteira)
{
    if (ModelState.IsValid)
    {
        carteira.Saldo = 0;
        _context.Add(carteira);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    return View(carteira);
}
```

Fonte: Elaborado pelos autores (2026).

5 DESENVOLVIMENTO DA SOLUÇÃO MOBILE (DESENVOLVIMENTO MOBILE)

A mobilidade computacional é um requisito indispensável para organizações que atuam diretamente nas comunidades, ginásios esportivos e postos de arrecadação. Para atender a essa demanda com máxima qualidade visual e desempenho nativo, foi desenvolvido o aplicativo móvel `vestindo_sonhos_mobile` utilizando o framework Flutter (Google) e a linguagem orientada a objetos Dart.

5.1 Arquitetura do aplicativo móvel em Flutter e Material Design 3

O aplicativo móvel foi estruturado no diretório `vestindo_sonhos_mobile/lib/` seguindo uma arquitetura limpa dividida em camadas de modelos (`models/`),

serviços de rede (`services/api_service.dart`), componentes reutilizáveis (`widgets/`) e telas reativas (`screens/`). No arquivo principal `main.dart`, definiu-se o tema global `ThemeData` com `useMaterial3: true`, paleta de cores institucional baseada na semente cromática `#7C6FE0` (violeta solidário), tipografia padronizada e barras superiores (`AppBar`) centralizadas com contraste branco sobre fundo violeta.

Para garantir funcionamento perfeito tanto em ambientes de desenvolvimento (emulador Android via IP especial `10.0.2.2:5000`) quanto em smartphones físicos conectados via Wi-Fi ao servidor da ONG, implementou-se na classe `ApiService` e no widget `ServerConfigDialog` um mecanismo dinâmico de descoberta e configuração de endereço IP: o usuário pode tocar no ícone de engrenagem na tela de login ou no menu lateral e informar o IP da máquina servidora na rede local (ex.: `http://192.168.1.100:5000`), que fica salvo permanentemente na memória do aparelho.

5.2 Telas principais e fluxo de navegação por perfil

O aplicativo contempla seis telas principais projetadas com layouts responsivos que previnem estouros de pixels (*layout overflow*) em diferentes resoluções e densidades de tela:

- 1. Tela de Autenticação (`login_screen.dart`): apresenta o logotipo e identidade visual da ONG Vestindo Sonhos, campos validados de e-mail e senha com alternância de visibilidade, seletor de servidor no canto superior direito e tratamento amigável de erros de conexão ou credenciais;
- 2. Tela de Troca de Senha Obrigatória (`trocar_senha_obrigatoria_screen.dart`): acionada automaticamente quando o back-end informa que o usuário está utilizando a senha temporária inicial (`exigirTrocaSenha: true`), exigindo a criação de uma nova senha segura de no mínimo 6 caracteres;
- 3. Tela Inicial / Home (`home_screen.dart`): disponível tanto para o perfil Administrador quanto para o perfil Colaborador, exibe o cabeçalho de boas-vindas com o nome do usuário logado, os quatro cartões da Visão Geral da ONG (Projetos Ativos, Voluntários, Itens em Estoque e Beneficiários) dispostos em linhas responsivas (`Row` + `Expanded`), as tags das Principais Necessidades da Comunidade (`#Calças`, `#Comida`, `#Roupas`,

`#Meias`) e os atalhos para os módulos. Conta ainda com botão direto de Sair (`Icons.logout`) na barra superior e menu lateral (`Drawer`) completo;

- 4. Tela de Inventário e Estoque (`inventario_screen.dart`): exibe painel superior com total de produtos e soma geral de unidades em estoque, barra de pesquisa instantânea por nome ou cor, filtros horizontais por categoria (`FilterChip`) e botão flutuante que permite lançar entradas/saídas de doações e gerenciar o catálogo de produtos e tipos de ativo;
- 5. Tela de Beneficiários e Dependentes (`beneficiarios_screen.dart`): apresenta a listagem de famílias com filtros por situação (`Todos`, `Ok`, `Aguardando`, `Bloqueado`), busca por nome ou CPF e ficha detalhada expansível que permite adicionar, editar ou remover tanto o beneficiário responsável quanto seus dependentes;
- 6. Tela Financeira (`financeiro_screen.dart`) e Tela de Eventos/Voluntários (`eventos_screen.dart`): a aba Financeira (visível apenas para Administradores) consolida o saldo total disponível, entradas, saídas, carteiras e permite lançar novas transações; já a aba de Eventos disponibiliza abas internas para gerenciar campanhas solidárias, alocar voluntários e administrar o cadastro de voluntários.

5.3 Autenticação JWT, sincronização de dados e consumo das APIs REST

A comunicação entre o aplicativo móvel e o back-end .NET 10 ocorre de forma assíncrona (`async`/`await` com `Future<T>`) utilizando o pacote oficial `http`. Após o login bem-sucedido, o token JWT recebido é armazenado localmente via `SharedPreferences`. Em todas as requisições subsequentes (`GET`, `POST`, `PUT`, `DELETE`), a classe `ApiService` injeta automaticamente o cabeçalho `'Authorization': 'Bearer $token'` e `'Content-Type': 'application/json; charset=UTF-8'`. Sempre que um registro é criado, editado ou excluído no celular, a tela recarrega imediatamente os dados da API REST, assegurando sincronia imediata com o banco de dados MySQL central.

6 ARQUITETURA DE SOFTWARE (PROGRAMAÇÃO APLICADA EM .NET)

A arquitetura de software define a estrutura fundamental de um sistema computacional, compreendendo seus elementos de software, as propriedades

externamente visíveis desses elementos e os relacionamentos entre eles (BASS; CLEMENTS; KAZMAN, 2021). Na construção da solução Vestindo Sonhos, aplicaram-se os conceitos avançados da disciplina de Programação Aplicada em .NET para criar uma arquitetura limpa, fortemente tipada, orientada a objetos e organizada em camadas.

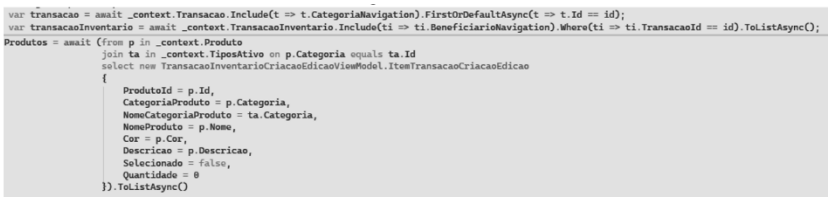
6.1 Orientação a objetos e princípios SOLID aplicados

A Programação Orientada a Objetos (POO) foi empregada de forma transversal em todo o ecossistema C# (.NET 10) e Dart (Flutter), garantindo modularidade, reuso e facilidade de manutenção:

- Encapsulamento e Proteção de Estado: as classes de modelo em `VestindoSonhos/Models/` encapsulam o estado das entidades persistentes por meio de propriedades autoimplementadas (`get; set;`) decoradas com Data Annotations (`[Required]`, `[StringLength]`, `[Column]`), garantindo que regras de integridade sintática sejam validadas antes mesmo de atingir o banco de dados;
- Herança e Especialização: no modelo de domínio e de banco de dados, aplicou-se herança conceitual na entidade `Transacao`, que centraliza os atributos comuns de auditoria (`Id`, `Usuario`, `Hora`, `Razao`, `Observacao`, `Categoria`) e é especializada pelas classes `TransacaoInventario` e `TransacaoFinanceira`. Na camada de controle, todas as classes Web herdam de `Controller` e as classes de API herdam de `ControllerBase`;
- Polimorfismo e Interfaces: o uso extensivo de interfaces da plataforma .NET — como `IActionResult`, `ICollection<T>`, `IAsyncActionFilter` (implementada em `ExigirTrocaSenhaFilter`) e `IConfiguration` — permite tratamento polimórfico de respostas HTTP e coleções de dados;
- Injeção de Dependência e Princípios SOLID: seguindo o Princípio da Inversão de Dependência (DIP) e o Princípio da Responsabilidade Única (SRP) de Martin (2020), nenhum controlador instancia diretamente o contexto de banco de dados com a palavra-chave `new`. Em vez disso, o contêiner de Injeção de Dependência do ASP.NET Core gerencia o ciclo de vida

(`Scoped`) da classe `VestindosonhosdbContext` e a fornece automaticamente nos construtores dos controladores.

Figura 4 – Consultas parametrizadas utilizando Entity Framework Core e LINQ



```
var transacao = await _context.Transacao.Include(t => t.CategoriaNavigation).FirstOrDefaultAsync(t => t.Id == id);
var transacaoInventario = await _context.TransacaoInventario.Include(ti => ti.BeneficiarioNavigation).Where(ti => ti.TransacaoId == id).ToListAsync();
Produtos = await (from p in _context.Produto
                  join ta in _context.TiposAtivo on p.Categoria equals ta.Id
                  select new TransacaoInventarioCriacaoEdicaoViewModel.ItemTransacaoCriacaoEdicao
                  {
                      ProdutoId = p.Id,
                      CategoriaProduto = p.Categoria,
                      NomeCategoriaProduto = ta.Categoria,
                      NomeProduto = p.Nome,
                      Cor = p.Cor,
                      Descricao = p.Descricao,
                      Selecionado = false,
                      Quantidade = 0
                  }).ToListAsync()
```

Fonte: Elaborado pelos autores (2026).

6.2 Organização em camadas, modularização, ViewModels e DTOs

Conforme preconiza Fowler (2018), a separação de um sistema corporativo em camadas lógicas evita o acoplamento acidental entre a interface de usuário e o esquema físico do banco de dados. No projeto Vestindo Sonhos, além das camadas de Apresentação (Web Razor e Mobile Flutter), Aplicação (Controllers MVC e API) e Persistência (EF Core e MySQL), implementou-se uma camada específica de objetos de transporte composta por 15 `ViewModels` (voltadas às telas Web complexas, como `DashboardViewModel`, `HomeFinanceiraViewModel`, `DetalhesBeneficiarioViewModel` e `DetalhesEventoViewModel`) e 7 conjuntos de `DTOs` (voltados aos contratos JSON das APIs REST). A Tabela 4 sintetiza a organização arquitetural em camadas da solução.

Tabela 4 – Arquitetura em camadas e distribuição modular da solução tecnológica Vestindo Sonhos

Camada Lógica	Tecnologia / Namespace	Artefatos Principais	Papel Arquitetural
Apresentação Web	ASP.NET Core Razor Views + CSS3 + Chart.js	Views/Home, Beneficiarios, Inventario, Produtos, Financeiro, Eventos, Voluntarios	Renderização server-side de páginas HTML5 responsivas para administração na sede.
Apresentação Mobile	Flutter SDK 3 + Dart (Material Design 3)	lib/screens/login, home, inventario, beneficiarios, financeiro, eventos	Aplicativo nativo Android/iOS para operação móvel em campo e campanhas.
Serviços e Controle (API & MVC)	C# .NET 10 (VestindoSonhos.Controllers)	12 Controladores MVC + 7 Controladores API RESTful + ExigirTrocaSenhaFilter	Roteamento HTTP, autenticação híbrida

Camada Lógica	Tecnologia / Namespace	Artefatos Principais	Papel Arquitetural
			(Cookie/JWT), autorização por Roles e orquestração.
Transferência de Dados	C# ViewModels & DTOs	VestindoSonhos.ViewModels (15 classes) e VestindoSonhos.Models.DTOs	Isolamento entre entidades do banco e contratos de tela/JSON, prevenindo Overposting.
Acesso a Dados (ORM)	Entity Framework Core 10 + LINQ	VestindosonhosdbContext.cs e Models/*.cs (14 entidades mapeadas)	Tradução de consultas LINQ tipadas em C# para comandos SQL parametrizados (anti-SQL Injection).
Persistência Relacional	MySQL Server 8.0 (InnoDB)	VESTINDOSONHOSDB (Tabelas, FK Constraints, Views, Triggers, Procedures)	Armazenamento transacional ACID e execução de regras automatizadas de saldo.

Fonte: Elaborada pelos autores (2026).

7 PROJETO E PROGRAMAÇÃO DO BANCO DE DADOS

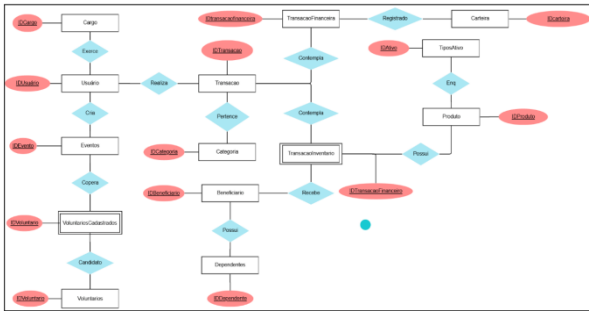
Um projeto de banco de dados corporativo bem estruturado é determinante para garantir consistência transacional, integridade referencial e alto desempenho analítico (ELMASRI; NAVATHE, 2019; DATE, 2004). Na solução tecnológica da ONG Vestindo Sonhos, o banco de dados `VESTINDOSONHOSDB` foi projetado para o SGBD MySQL 8.0, servindo simultaneamente como repositório único da verdade (*Single Source of Truth*) para o portal Web MVC, para as APIs RESTful e para o aplicativo móvel Flutter.

7.1 Modelo conceitual (MER) e modelo lógico normalizado (3FN)

A modelagem conceitual (Modelo Entidade-Relacionamento – MER) identificou as entidades fundamentais do domínio da ONG e suas cardinalidades relacionais: um `Cargo` (`1:N`) classifica múltiplos registros de `Usuario`; um `Beneficiario` responsável (`1:N`) possui zero ou vários registros de `Dependente`; um `Usuario` responsável (`1:N`) coordena múltiplos `Eventos`, e cada evento

relaciona-se em cardinalidade muitos-para-muitos (`N:N`) com `Voluntario` por meio da entidade associativa `VoluntarioAtribuido`. No domínio de estoque, um `TiposAtivo` (`1:N`) agrupa diversos itens de `Produto`, cada qual mantendo um relacionamento um-para-um (`1:1`) com seu saldo consolidado na entidade `Inventario`. Por fim, no núcleo transacional, `Usuario` (`1:N`) e `Categoria` (`1:N`) originam registros na entidade generalizada `Transacao`, que se especializa (`1:1`) em `TransacaoInventario` (vinculada a `Produto` e opcionalmente a `Beneficiario`) ou em `TransacaoFinanceira` (vinculada a `Carteira`).

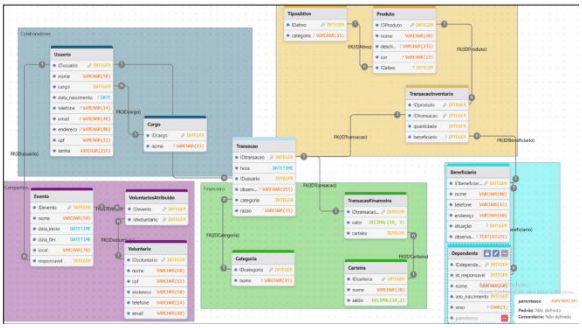
Figura 5 – Modelo Conceitual Entidade-Relacionamento (MER) do banco de dados



Fonte: Elaborado pelos autores (2026).

No modelo lógico relacional, todas as estruturas foram submetidas ao processo de normalização até a Terceira Forma Normal (3FN): (a) na Primeira Forma Normal (1FN), eliminaram-se atributos multivalorados e compostos, garantindo atomicidade em todas as colunas; (b) na Segunda Forma Normal (2FN), assegurou-se que todos os atributos não-chave dependam integralmente da chave primária, inclusive na tabela de chave composta `VoluntarioAtribuido(evento, voluntario)`, onde o atributo `data_adicao` depende do par `(evento, voluntario)`; e (c) na Terceira Forma Normal (3FN), removeram-se dependências transitivas ao isolar nomes de cargos, nomes de categorias contábeis e descrições de tipos de ativos nas tabelas `Cargos`, `Categoria` e `TiposAtivo`.

Figura 6 – Modelo Lógico Relacional (3FN) das tabelas do banco VESTINDOSONHOSDB



Fonte: Elaborado pelos autores (2026).

7.2 Modelo físico e dicionário das tabelas do MySQL 8.0

O modelo físico foi implementado utilizando o mecanismo de armazenamento transacional `InnoDB` (que oferece suporte completo às propriedades ACID — Atomicidade, Consistência, Isolamento e Durabilidade — e bloqueio em nível de linha) e o conjunto de caracteres `utf8mb4` com colação `utf8mb4_unicode_ci`. A Tabela 5 detalha o esquema físico completo das tabelas que compõem a base `VESTINDOSONHOSDB`.

Tabela 5 – Especificação do modelo físico das tabelas do banco de dados VESTINDOSONHOSDB

Tabela	Colunas e Tipos Físicos (MySQL 8.0)	Restrições de Integridade (PK / FK / Unique)
Cargos	id INT AUTO_INCREMENT, nome VARCHAR(50) NOT NULL	PK(id)
Usuario	id INT AUTO_INCREMENT, nome VARCHAR(50), data_nascimento DATE, telefone VARCHAR(20), email VARCHAR(40), endereco VARCHAR(50), cpf VARCHAR(15), cargo INT, senha VARCHAR(255), ExigirTrocaSenha TINYINT(1)	PK(id), UNIQUE(email), UNIQUE(cpf), FK(cargo) REFERENCES Cargos(id)
Beneficiario	id INT AUTO_INCREMENT, nome VARCHAR(60), cpf VARCHAR(15), telefone VARCHAR(20), endereco VARCHAR(60), observacao VARCHAR(255), situacao VARCHAR(20)	PK(id)
Dependente	id INT AUTO_INCREMENT, id_responsavel INT, nome VARCHAR(60), idade INT DEFAULT 0, ano_nascimento INT, sexo CHAR(1), parentesco VARCHAR(20), observacao VARCHAR(255)	PK(id), FK(id_responsavel) REFERENCES Beneficiario(id) ON DELETE CASCADE

Tabela	Colunas e Tipos Físicos (MySQL 8.0)	Restrições de Integridade (PK / FK / Unique)
Voluntario	id INT AUTO_INCREMENT, nome VARCHAR(60), cpf VARCHAR(15), endereco VARCHAR(60), telefone VARCHAR(20), email VARCHAR(40)	PK(id)
Eventos	id INT AUTO_INCREMENT, nome VARCHAR(60), data_inicio DATETIME, data_fim DATETIME, local VARCHAR(60), observacao VARCHAR(255), responsavel INT	PK(id), FK(responsavel) REFERENCES Usuario(id)
VoluntarioAtribuido	evento INT NOT NULL, voluntario INT NOT NULL, data_adicao DATETIME DEFAULT CURRENT_TIMESTAMP	PK(evento, voluntario), FK(evento) -> Eventos(id), FK(voluntario) -> Voluntario(id) CASCADE
TiposAtivo	id INT AUTO_INCREMENT, categoria VARCHAR(30) NOT NULL	PK(id)
Produto	id INT AUTO_INCREMENT, nome VARCHAR(30), descricao VARCHAR(100), tamanho VARCHAR(15), cor VARCHAR(20), categoria INT	PK(id), FK(categoria) REFERENCES TiposAtivo(id)
Inventario	id INT NOT NULL, quantidade INT NOT NULL DEFAULT 0	PK(id), FK(id) REFERENCES Produto(id) ON DELETE CASCADE
Carteira	id INT AUTO_INCREMENT, nome VARCHAR(30) NOT NULL, saldo DECIMAL(10,2) DEFAULT 0.00	PK(id)
Categoria	id INT AUTO_INCREMENT, nome VARCHAR(30) NOT NULL	PK(id)
Transacao	id INT AUTO_INCREMENT, usuario INT, hora DATETIME, razao VARCHAR(20), observacao VARCHAR(100), categoria INT	PK(id), FK(usuario) -> Usuario(id), FK(categoria) -> Categoria(id)
TransacaoInventario	id INT NOT NULL, produto INT NOT NULL, quantidade INT NOT NULL, beneficiario INT NULL	PK(id), FK(id) -> Transacao(id), FK(produto) -> Produto(id), FK(beneficiario) -> Beneficiario(id)
TransacaoFinanceira	id INT NOT NULL, valor DECIMAL(10,2) NOT NULL, carteira INT NOT NULL	PK(id), FK(id) -> Transacao(id), FK(carteira) -> Carteira(id)

Fonte: Elaborada pelos autores (2026).

7.3 Stored Procedures, Triggers e Views implementadas

Para elevar a segurança transacional e o desempenho de consultas agregadas, foram programados diretamente no SGBD quatro objetos avançados de

banco de dados: (1) a View `Financeiro`, que realiza o `INNER JOIN` permanente entre `Transacao` e `TransacaoFinanceira`, expondo uma visão contábil unificada; (2) a Stored Procedure `sp_RelatorioFinanceiroResumido(IN p_ano INT)`, que consolida no próprio servidor de banco de dados as somas mensais de receitas (`Entrada`/`Doação`), despesas, saldo líquido e volume de transações de um exercício anual; (3) a Stored Procedure `sp_GarantirColunasDependente()`, que utiliza SQL dinâmico (`PREPARE`/`EXECUTE`) sobre o `INFORMATION_SCHEMA` para garantir migração idempotente das colunas da tabela `Dependente`; e (4) a Trigger `trg_AtualizaEstoque` (`AFTER INSERT ON TransacaoInventario`), que intercepta cada lançamento de entrada ou saída de produto e recalcula automaticamente o saldo na tabela `Inventario`, impedindo saldos negativos por meio da função `GREATEST(0, quantidade - NEW.quantidade)`.

7.4 Script completo do banco de dados (MySQL 8.0)

Em cumprimento ao requisito explícito da Etapa 7 do Manual do PIM IV (página 27), apresenta-se a seguir o Script SQL completo (`ScriptVestindoSonhos_MySQL.sql`) utilizado de forma compartilhada pelas APIs REST, pela aplicação Web MVC e pelo aplicativo Mobile Flutter:

```
-- PROJETO PIM - ONG VESTINDO SONHOS
-- Script DDL & Recursos Avançados Adaptados para MySQL (MySQL Workbench)
CREATE DATABASE IF NOT EXISTS VESTINDOSONHOSDB;
USE VESTINDOSONHOSDB;
-- ------------------------------------------------------------------------------
-- 1. CRIAÇÃO DAS TABELAS
-- ------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS Cargos (
    id INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(50) NOT NULL,
    PRIMARY KEY (id)
);
CREATE TABLE IF NOT EXISTS TiposAtivo (
    id INT NOT NULL AUTO_INCREMENT,
    categoria VARCHAR(15) NOT NULL UNIQUE,
    PRIMARY KEY (id)
);
CREATE TABLE IF NOT EXISTS Usuario (
    id INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(50) NOT NULL,
    cargo INT NOT NULL,
    data_nascimento DATE,
    telefone VARCHAR(14),
    email VARCHAR(40) UNIQUE,
    endereco VARCHAR(80),
    cpf VARCHAR(11) NOT NULL UNIQUE,
    senha VARCHAR(255) NOT NULL,
    exigir_troca_senha TINYINT(1) NOT NULL DEFAULT 0,
    PRIMARY KEY (id),
    CONSTRAINT FK_Usuario_Cargo FOREIGN KEY (cargo) REFERENCES Cargos(id)
);
CREATE TABLE IF NOT EXISTS Eventos (
    id INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(50) NOT NULL,
    data_inicio DATETIME NOT NULL,
    data_fim DATETIME NOT NULL,
    local VARCHAR(70),
    observacao VARCHAR(255),
    responsavel INT NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT FK_Evento_Usuario FOREIGN KEY (responsavel) REFERENCES Usuario(id)
);
CREATE TABLE IF NOT EXISTS Voluntario (
    id INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(60) NOT NULL,
    cpf VARCHAR(11) NOT NULL,
    endereco VARCHAR(50),
```

```
    telefone VARCHAR(14),
    email VARCHAR(40),
    PRIMARY KEY (id)
);
CREATE TABLE IF NOT EXISTS Beneficiario (
    id INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(80) NOT NULL,
    telefone VARCHAR(15) NOT NULL,
    endereco VARCHAR(60) NOT NULL,
    situacao VARCHAR(20),
    observacao TEXT,
    PRIMARY KEY (id)
);
CREATE TABLE IF NOT EXISTS VoluntarioAtribuido (
    evento INT NOT NULL,
    voluntario INT NOT NULL,
    data_adicao DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (evento, voluntario),
    CONSTRAINT FK_VA_Evento FOREIGN KEY (evento) REFERENCES Eventos(id),
    CONSTRAINT FK_VA_Voluntario FOREIGN KEY (voluntario) REFERENCES Voluntario(id)
);
CREATE TABLE IF NOT EXISTS Categoria (
    id INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(15) NOT NULL,
    PRIMARY KEY (id)
);
CREATE TABLE IF NOT EXISTS Carteira (
    id INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(30) NOT NULL,
    saldo DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (id)
);
CREATE TABLE IF NOT EXISTS Transacao (
    id INT NOT NULL AUTO_INCREMENT,
    hora DATETIME,
    usuario INT NOT NULL,
    observacao VARCHAR(255),
    categoria INT NOT NULL,
    razao VARCHAR(255),
    PRIMARY KEY (id),
    CONSTRAINT FK_Transacao_Usuario FOREIGN KEY (usuario) REFERENCES Usuario(id),
    CONSTRAINT FK_Transacao_Categoria FOREIGN KEY (categoria) REFERENCES Categoria(id)
);
CREATE TABLE IF NOT EXISTS TransacaoFinanceira (
    id INT NOT NULL,
    valor DECIMAL(10, 2) NOT NULL,
    carteira INT NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT FK_TF_Transacao FOREIGN KEY (id) REFERENCES Transacao(id),
    CONSTRAINT FK_TF_Carteira FOREIGN KEY (carteira) REFERENCES Carteira(id)
);
CREATE TABLE IF NOT EXISTS Produto (
    id INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(60) NOT NULL,
    descricao VARCHAR(255) DEFAULT '',
    cor VARCHAR(15) DEFAULT '',
    categoria INT,
    PRIMARY KEY (id),
    CONSTRAINT FK_Produto_Tipo FOREIGN KEY (categoria) REFERENCES TiposAtivo(id)
);
CREATE TABLE IF NOT EXISTS TransacaoInventario (
    produto_id INT NOT NULL,
    transacao_id INT NOT NULL,
    quantidade INT NOT NULL,
    beneficiario INT,
    PRIMARY KEY (produto_id, transacao_id),
    CONSTRAINT FK_TI_Produto FOREIGN KEY (produto_id) REFERENCES Produto(id),
    CONSTRAINT FK_TI_Transacao FOREIGN KEY (transacao_id) REFERENCES Transacao(id),
    CONSTRAINT FK_TI_Beneficiario FOREIGN KEY (beneficiario) REFERENCES Beneficiario(id)
);
CREATE TABLE IF NOT EXISTS Dependente (
    id INT NOT NULL AUTO_INCREMENT,
    id_responsavel INT NOT NULL,
    nome VARCHAR(60) NOT NULL,
    idade INT NOT NULL DEFAULT 0,
    ano_nascimento INT DEFAULT 0,
    sexo CHAR(1) NOT NULL,
    parentesco VARCHAR(20),
    observacao VARCHAR(255),
    PRIMARY KEY (id),
    CONSTRAINT FK_Dependente_Beneficiario FOREIGN KEY (id_responsavel) REFERENCES
Beneficiario(id)
);
DELIMITER //
DROP PROCEDURE IF EXISTS sp_GarantirColunasDependente //
CREATE PROCEDURE sp_GarantirColunasDependente()
BEGIN
    IF NOT EXISTS (SELECT 1 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = DATABASE()
AND TABLE_NAME = 'Dependente' AND COLUMN_NAME = 'idade') THEN
        ALTER TABLE Dependente ADD COLUMN idade INT NOT NULL DEFAULT 0;
    END IF;
    IF NOT EXISTS (SELECT 1 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = DATABASE()
AND TABLE_NAME = 'Dependente' AND COLUMN_NAME = 'observacao') THEN
        ALTER TABLE Dependente ADD COLUMN observacao VARCHAR(255) NULL;
    END IF;
    IF EXISTS (SELECT 1 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = DATABASE() AND
TABLE_NAME = 'Dependente' AND COLUMN_NAME = 'ano_nascimento') THEN
        ALTER TABLE Dependente MODIFY COLUMN ano_nascimento INT NULL DEFAULT 0;
    END IF;
END //
```

```
DELIMITER ;
CALL sp_GarantirColunasDependente();
DROP PROCEDURE IF EXISTS sp_GarantirColunasDependente;
-- ------------------------------------------------------------------------------
-- 2. TABELA PRINCIPAL DE INVENTÁRIO (Controle Consolidado de Estoque)
-- ------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS Inventario (
    id INT NOT NULL AUTO_INCREMENT,
    produto_id INT NOT NULL UNIQUE,
    quantidade INT NOT NULL DEFAULT 0,
    ultima_atualizacao DATETIME DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    CONSTRAINT FK_Inventario_Produto FOREIGN KEY (produto_id) REFERENCES Produto(id) ON
DELETE CASCADE
);
-- Inicializa o inventário para produtos existentes caso ainda não estejam mapeados
INSERT IGNORE INTO Inventario (produto_id, quantidade, ultima_atualizacao)
SELECT p.id, 0, NOW()
FROM Produto p;
-- ------------------------------------------------------------------------------
-- 3. VIEW: Financeiro (Visão unificada para o módulo financeiro)
-- ------------------------------------------------------------------------------
CREATE OR REPLACE VIEW Financeiro AS
SELECT
    id,
    hora,
    usuario,
    observacao,
    categoria,
    razao
FROM Transacao;
-- ------------------------------------------------------------------------------
-- 4. STORED PROCEDURE: sp_RelatorioFinanceiroResumido
-- Retorna Entradas, Saídas e Saldo do mês atual fazendo JOIN entre
-- TransacaoFinanceira e Financeiro
-- ------------------------------------------------------------------------------
DELIMITER //
DROP PROCEDURE IF EXISTS sp_RelatorioFinanceiroResumido //
CREATE PROCEDURE sp_RelatorioFinanceiroResumido(
    IN p_Mes INT,
    IN p_Ano INT
)
BEGIN
    -- Se não especificado (NULL), assume o mês e ano atuais
    SET p_Mes = COALESCE(p_Mes, MONTH(CURRENT_DATE()));
    SET p_Ano = COALESCE(p_Ano, YEAR(CURRENT_DATE()));
    SELECT
        p_Mes AS MesReferencia,
        p_Ano AS AnoReferencia,
        COALESCE(SUM(CASE WHEN f.razao = 'Entrada' THEN tf.valor ELSE 0.00 END), 0.00) AS
TotalEntradas,
        COALESCE(SUM(CASE WHEN f.razao IN ('Saida', 'Saída') THEN tf.valor ELSE 0.00 END),
0.00) AS TotalSaidas,
        COALESCE(SUM(CASE WHEN f.razao = 'Entrada' THEN tf.valor ELSE -tf.valor END), 0.00)
AS SaldoLiquido,
        COUNT(tf.id) AS QuantidadeTransacoes
    FROM TransacaoFinanceira tf
    INNER JOIN Financeiro f ON tf.id = f.id
    WHERE MONTH(f.hora) = p_Mes
      AND YEAR(f.hora) = p_Ano;
END //
DELIMITER ;
-- ------------------------------------------------------------------------------
-- 5. TRIGGER: trg_AtualizaEstoque
-- Atualiza a quantidade na tabela Inventario ao inserir em TransacaoInventario
-- ------------------------------------------------------------------------------
DELIMITER //
DROP TRIGGER IF EXISTS trg_AtualizaEstoque //
CREATE TRIGGER trg_AtualizaEstoque
AFTER INSERT ON TransacaoInventario
FOR EACH ROW
BEGIN
    DECLARE v_razao VARCHAR(255);
    -- 1. Garante que o produto existe na tabela Inventario
    INSERT IGNORE INTO Inventario (produto_id, quantidade, ultima_atualizacao)
    VALUES (NEW.produto_id, 0, NOW());
    -- 2. Busca a razão ('Entrada' ou 'Saida') na tabela de transações
    SELECT razao INTO v_razao
    FROM Transacao
    WHERE id = NEW.transacao_id
    LIMIT 1;
    -- 3. Atualiza o estoque conforme o tipo da movimentação
    IF v_razao = 'Entrada' THEN
        UPDATE Inventario
        SET quantidade = quantidade + NEW.quantidade,
            ultima_atualizacao = NOW()
        WHERE produto_id = NEW.produto_id;
    ELSEIF v_razao IN ('Saida', 'Saída') THEN
        UPDATE Inventario
        SET quantidade = quantidade - NEW.quantidade,
            ultima_atualizacao = NOW()
        WHERE produto_id = NEW.produto_id;
    END IF;
END //
DELIMITER ;
-- ------------------------------------------------------------------------------
-- 6. VIEWS DE COMPATIBILIDADE COM O ENTITY FRAMEWORK CORE
-- Garante funcionamento independente se a consulta do EF Core usar singular ou plural
-- ------------------------------------------------------------------------------
CREATE OR REPLACE VIEW cargo AS SELECT * FROM Cargos;
```

```
CREATE OR REPLACE VIEW evento AS SELECT * FROM Eventos;
CREATE OR REPLACE VIEW tipos_ativo AS SELECT * FROM TiposAtivo;
CREATE OR REPLACE VIEW voluntario_atribuido AS SELECT * FROM VoluntarioAtribuido;
-- ------------------------------------------------------------------------------
-- 7. DADOS INICIAIS (SEEDS) PARA INICIALIZAÇÃO DO SISTEMA
-- Insere registros essenciais para testes e acesso com usuário Admin
-- ------------------------------------------------------------------------------
-- Cargos
INSERT IGNORE INTO Cargos (id, nome) VALUES
(1, 'Administrador'),
(2, 'Colaborador');
-- Categorias de itens
INSERT IGNORE INTO TiposAtivo (id, categoria) VALUES
(1, 'Roupas'),
(2, 'Calçados'),
(3, 'Alimentos');
-- Usuários do Sistema para testes e apresentação (Todos com Senha inicial: 123456)
INSERT IGNORE INTO Usuario (id, nome, cargo, cpf, email, telefone, senha) VALUES
(1, 'Admin Teste', 1, '12345678901', 'admin@teste.org', '(11) 98765-4321', '123456'),
(2, 'Mariana Colaboradora', 2, '23456789012', 'mariana@teste.org', '(11) 97654-3210',
'123456'),
(3, 'Carlos Colaborador', 2, '34567890123', 'carlos@teste.org', '(11) 96543-2109', '123456');
-- Carteiras financeiras (Essenciais para registro financeiro)
INSERT IGNORE INTO Carteira (id, nome, saldo) VALUES
(1, 'Conta Principal (Banco)', 5000.00),
(2, 'Caixa Físico (Sede)', 1500.00),
(3, 'Fundo de Reserva', 10000.00);
-- Categorias financeiras
INSERT IGNORE INTO Categoria (id, nome) VALUES
(1, 'Doação'),
(2, 'Eventos'),
(3, 'Campanhas'),
(4, 'Despesas Operacionais');
-- Voluntários para teste
INSERT IGNORE INTO Voluntario (id, nome, cpf, endereco, telefone, email) VALUES
(1, 'Carlos Eduardo Lima', '11122233344', 'Rua das Flores, 120', '11988887777',
'carlos.lima@ong.org'),
(2, 'Mariana Santos Silva', '55566677788', 'Av. Paulista, 1500', '11977776666',
'mariana.santos@ong.org');
-- Eventos iniciais
INSERT IGNORE INTO Eventos (id, nome, data_inicio, data_fim, local, responsavel, observacao)
VALUES
(1, 'Campanha do Agasalho 2026', '2026-06-01 09:00:00', '2026-06-10 18:00:00', 'Ginásio
Municipal', 1, 'Arrecadação de casacos e cobertores'),
(2, 'Bazar Beneficente de Outono', '2026-10-15 09:00:00', '2026-10-15 17:00:00', 'Sede
Central da ONG', 1, 'Venda comunitária e distribuição');
-- Associação inicial
INSERT IGNORE INTO VoluntarioAtribuido (evento, voluntario, data_adicao) VALUES
(1, 1, NOW()),
(1, 2, NOW());
```

8 INFRAESTRUTURA EM NUVEM E DEVOPS (CLOUD COMPUTING E DEVOPS)

A disciplina de Cloud Computing e DevOps preconiza que o ciclo de vida moderno de engenharia de software não se encerra na codificação: é imprescindível estabelecer uma infraestrutura elástica em nuvem e automatizar os processos de compilação, verificação e implantação (KIM et al., 2021). Neste capítulo, apresenta-se o plano de execução para a infraestrutura em nuvem e a esteira CI/CD implementada no projeto Vestindo Sonhos.

8.1 Arquitetura em nuvem, serviços utilizados e containers Docker

A arquitetura de implantação em nuvem da solução Vestindo Sonhos foi desenhada sob o paradigma Cloud-Native utilizando containers Docker e serviços gerenciados de Plataforma como Serviço (PaaS), garantindo portabilidade entre provedores como Microsoft Azure e Amazon Web Services (AWS), além de permitir execução híbrida em servidor local da ONG. Nessa topologia, o tráfego externo HTTPS passa por uma camada de proteção de borda (CDN/WAF), alcança o serviço de computação em container que executa o binário ASP.NET Core (.NET 10) e

persiste os dados em uma instância gerenciada de MySQL 8.0 em sub-rede privada. A Tabela 6 detalha os componentes dessa infraestrutura.

Tabela 6 – Especificação da arquitetura em nuvem, containers e serviços para a solução Vestindo Sonhos

Componente Arquitetural	Serviço em Nuvem Especificado	Configuração Técnica e Finalidade
Camada de Borda, DNS e Segurança	Cloudflare DNS + WAF / Azure Front Door	Certificado SSL/TLS 1.3 automático (HTTPS), mitigação de ataques DDoS e proteção OWASP Top 10 para a Web e APIs REST.
Serviço de Computação (Web + API)	Azure App Service for Containers / AWS ECS Fargate	Execução do container Linux com runtime .NET 10 (mcr.microsoft.com/dotnet/aspnet:10.0), escutando na porta 8080/5000.
Banco de Dados Gerenciado (DBaaS)	Azure Database for MySQL 8.0 / Amazon RDS for MySQL	Servidor MySQL 8.0 gerenciado com alta disponibilidade, backups automáticos diários (retenção de 7 dias) e criptografia em repouso.
Registro de Containers e Artefatos	GitHub Container Registry (GHCR) & GitHub Actions Artifacts	Armazenamento versionado das imagens Docker, do pacote portátil Windows e do instalador Android (VestindoSonhos_Mobile.apk).
Observabilidade e Monitoramento	Azure Application Insights / Amazon CloudWatch + HealthChecks	Coleta de logs estruturados (ILogger), métricas de uso de CPU/RAM, tempo de resposta das APIs e alertas de erro 5xx.

Fonte: Elaborada pelos autores (2026).

8.2 Pipeline CI/CD com GitHub Actions, monitoramento, escalabilidade e segurança

A esteira de Integração Contínua e Entrega Contínua (CI/CD) foi implementada no arquivo `.github/workflows/deploy.yml` do repositório GitHub oficial do projeto (`https://github.com/vinicius311006/projeto-pim-sistema-ong-main`). Sempre que um desenvolvedor realiza um `push` ou `pull_request` na branch `main`, o GitHub Actions provisiona uma máquina virtual `ubuntu-latest` e executa automaticamente os passos de checkout do código, instalação do .NET 10.0 SDK, restauração de pacotes (`dotnet restore`), compilação sem erros em modo `Release` (`dotnet build --configuration Release`), publicação da aplicação (`dotnet publish`) e arquivamento do pacote de implantação (`upload-artifact`).

Em termos de escalabilidade e segurança de infraestrutura, o uso de autenticação *stateless* via JWT nas APIs REST permite escalabilidade horizontal imediata (*auto-scaling* de 1 para N instâncias de container conforme o aumento de

acessos simultâneos durante grandes campanhas solidárias), enquanto o isolamento do banco de dados em VNet privada, o uso de variáveis de ambiente seguras para segredos e a comunicação criptografada TLS asseguram proteção integral contra interceptação de dados.

9 GERENCIAMENTO ÁGIL DO PROJETO

A complexidade inerente ao desenvolvimento simultâneo de uma plataforma Web MVC, uma camada de APIs RESTful, um aplicativo móvel em Flutter e um banco de dados relacional exigiu a adoção de metodologias ágeis de gestão de projetos. Fundamentado no Guia do Scrum (SCHWABER; SUTHERLAND, 2020) combinado ao método Kanban, o gerenciamento do projeto priorizou entregas incrementais funcionais, inspeção contínua e rápida adaptação a mudanças.

9.1 Framework Scrum/Kanban, Product Backlog e Sprint Backlog

No início do semestre, os requisitos funcionais e não funcionais levantados para a ONG Vestindo Sonhos foram organizados em um *Product Backlog* unificado. Cada item foi descrito no formato de História de Usuário (*"Como [papel], desejo [ação], para [benefício]"*), classificado por prioridade de negócio e estimado em *Story Points* pela técnica de *Planning Poker*. O trabalho foi distribuído em quatro *Sprints* de duas semanas cada, compondo o *Sprint Backlog* detalhado na Tabela 7.

Tabela 7 – Product Backlog e Sprint Backlog do projeto tecnológico ONG Vestindo Sonhos

ID	Épico	História de Usuário / Item do Backlog	Prioridade	Story Points	Sprint Alocada
US01	Banco de Dados	Modelar e implementar o banco VESTINDOSONHOSDB no MySQL 8.0 com 14 tabelas normalizadas, View, Stored Procedures e Trigger de estoque.	Alta	8	Sprint 1
US02	Autenticação & Segurança	Implementar login híbrido (Cookie Web + JWT API), criptografia PBKDF2, perfis Admin/Colaborador e fluxo obrigatório de troca de senha inicial.	Alta	8	Sprint 1
US03	Módulo Beneficiários Web	Desenvolver o CRUD Web de Beneficiários e o gerenciamento integrado de Dependentes (com idade, sexo, parentesco e observação).	Alta	8	Sprint 2
US04	Módulo Inventário &	Desenvolver o controle Web de Inventário, catálogo de Produtos, Tipos	Alta	8	Sprint 2

ID	Épico	História de Usuário / Item do Backlog	Prioridade	Story Points	Sprint Alocada
	Produtos	de Ativo, baixa de estoque e exportação de relatórios CSV/Excel.			
US05	Módulos Financeiro & Eventos	Construir o módulo Financeiro (Carteiras, Categorias, Doações e Caixa) e o módulo de Eventos com atribuição de Voluntários.	Média	8	Sprint 2
US06	Camada de APIs RESTful	Construir os 7 controladores de API REST JSON (/api/auth, /dashboard, /beneficiarios, /inventario, /financeiro, /eventos, /usuarios).	Alta	8	Sprint 3
US07	Aplicativo Mobile Flutter	Desenvolver o app Flutter Material 3 com telas de Login, Home responsiva (Admin e Colaborador), Inventário, Beneficiários, Financeiro e Eventos.	Alta	13	Sprint 3
US08	DevOps, Pacote Portátil e ABNT	Configurar pipeline CI/CD no GitHub Actions, gerar executável portátil Windows, compilar APK Release e elaborar documentação ABNT PIM IV.	Alta	5	Sprint 4

Fonte: Elaborada pelos autores (2026).

9.2 Cronograma das Sprints, Quadro Kanban e definição das entregas

Para monitorar o fluxo de trabalho diário e evitar gargalos entre o desenvolvimento back-end e mobile, a equipe utilizou um Quadro Kanban dividido em quatro estágios: *A Fazer (To Do)*, *Em Desenvolvimento (In Progress)*, *Em Revisão e Testes Integrados (Testing/QA)* e *Concluído (Done)*. A Tabela 8 apresenta o cronograma executivo das Sprints, o estado final do Quadro Kanban e os artefatos concretos entregues em cada ciclo.

Tabela 8 – Cronograma executivo das Sprints, Quadro Kanban e entregas validadas

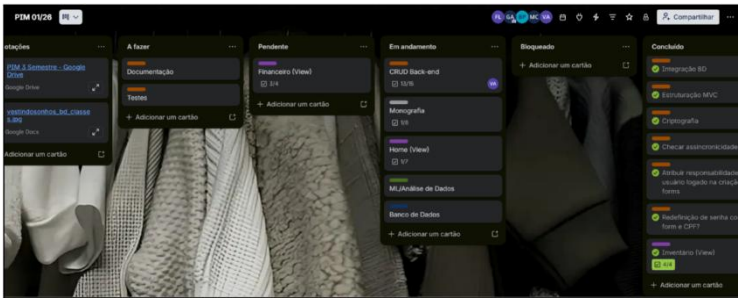
Sprint	Período de Execução	Quadro Kanban – Itens Validados em DONE	Entregáveis Incrementais Homologados
Sprint 1	Semanas 1 a 2	[DONE] US01 (MySQL 8.0 + Trigger + SPs) [DONE] US02 (SmartAuth Cookie/JWT + PBKDF2)	Banco de dados operacional, script de importação automática e sistema de autenticação seguro.
Sprint 2	Semanas 3 a 4	[DONE] US03 (Beneficiários e Dependentes) [DONE] US04 (Inventário e Produtos) [DONE] US05 (Financeiro, Carteiras e Eventos)	Portal Web MVC 100% funcional em todos os módulos administrativos e operacionais.
Sprint	Semanas 5	[DONE] US06 (Suíte completa de	Integração completa Web-API-Mobile,

Sprint	Período de Execução	Quadro Kanban – Itens Validados em DONE	Entregáveis Incrementais Homologados
3	a 6	APIs REST) [DONE] US07 (App Mobile Flutter Android/iOS)	app com Home e botão Sair para Colaborador e Admin, sem overflows.
Sprint 4	Semanas 7 a 8	[DONE] US08 (GitHub Actions CI/CD + Executável Portátil + APK Release + Relatório ABNT)	Pacote VestindoSonhos_Portavel (.exe + .apk), pipeline CI/CD ativo e documentação ABNT concluída.

Fonte: Elaborada pelos autores (2026).

Todas as entregas passaram por uma bateria rigorosa de validação automatizada e manual (Definição de Pronto — *Definition of Done*), verificando-se o retorno `200 OK` em todas as rotas autenticadas do portal Web e das APIs REST, a preservação de saldos de carteiras e estoque, a compilação limpa do pacote Android `VestindoSonhos_Mobile.apk` em modo Release e a sincronização integral no repositório GitHub.

Figura 7 – Quadro Kanban utilizado pela equipe no gerenciamento ágil do projeto



Fonte: Elaborado pelos autores (2026).

CONCLUSÃO

A realização deste Projeto Integrado Multidisciplinar IV (PIM IV) cumpriu integralmente o propósito de desenvolver e implantar uma solução tecnológica corporativa integrada capaz de apoiar a transformação digital da ONG Vestindo Sonhos. Ao articular na prática todas as disciplinas do quarto semestre do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade Paulista (UNIP), o projeto converteu os levantamentos teóricos anteriores em um ecossistema de software completo, seguro, responsivo e pronto para operação real.

Retomando os objetivos traçados na Introdução, verificou-se que cada etapa contribuiu de forma decisiva para o resultado alcançado: a caracterização organizacional e o planejamento empreendedor demonstraram como um sistema de código aberto e custo zero de licenciamento pode eliminar a desorganização de planilhas manuais no terceiro setor; a incorporação dos preceitos de responsabilidade social, relações étnico-raciais e acessibilidade digital assegurou um atendimento justo, inclusivo e respeitoso à diversidade das famílias vulneráveis; e a engenharia integrada do portal Web em ASP.NET Core MVC (.NET 10), das APIs RESTful com autenticação híbrida (Cookies e JWT) e do aplicativo móvel em Flutter (Material Design 3) proporcionou mobilidade total aos voluntários e colaboradores em campo.

De igual modo, a arquitetura de software orientada a objetos em camadas e o projeto do banco de dados relacional MySQL 8.0 — enriquecido com Triggers transacionais de estoque, Stored Procedures financeiras e Views — garantiram integridade referencial absoluta e rapidez nas consultas analíticas do Dashboard. Por fim, o plano de infraestrutura em nuvem com containers Docker, a automação CI/CD via GitHub Actions e o gerenciamento ágil com Scrum e Kanban consolidaram a maturidade técnica da entrega. Como perspectivas de evolução futura, destacam-se a integração de notificações *push* para convocação imediata de voluntários e um módulo de doação espontânea via PIX com conciliação bancária automática.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. **Software architecture in practice**. 4. ed. Boston: Addison-Wesley Professional, 2021.

BRASIL. **Lei nº 10.639, de 9 de janeiro de 2003**. Altera a Lei nº 9.394, de 20 de dezembro de 1996, que estabelece as diretrizes e bases da educação nacional, para incluir no currículo oficial da Rede de Ensino a obrigatoriedade da temática "História e Cultura Afro-Brasileira". Brasília, DF: Presidência da República, 2003.

BRASIL. **Lei nº 12.288, de 20 de julho de 2010**. Institui o Estatuto da Igualdade Racial. Brasília, DF: Presidência da República, 2010.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

CERVO, Amado Luiz; BERVIAN, Pedro Alcino; SILVA, Roberto da. **Metodologia científica**. 6. ed. São Paulo: Pearson Prentice Hall, 2007.

DATE, Christopher J. **Introdução a sistemas de bancos de dados**. 8. ed. Rio de Janeiro: Elsevier, 2004.

DORNELAS, José Carlos Assis. **Empreendedorismo: transformando ideias em negócios**. 8. ed. São Paulo: Empreende, 2021.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson Education do Brasil, 2019.

FOWLER, Martin. **Padrões de arquitetura de aplicações corporativas**. Porto Alegre: Bookman, 2018.

GIL, Antônio Carlos. **Métodos e técnicas de pesquisa social**. 7. ed. São Paulo: Atlas, 2019.

KIM, Gene; HUMBLE, Jez; DEBOIS, Patrick; WILLIS, John. **Manual de DevOps: como obter agilidade, confiabilidade e segurança em organizações tecnológicas**. Rio de Janeiro: Alta Books, 2021.

LAKATOS, Eva Maria; MARCONI, Marina de Andrade. **Fundamentos de metodologia científica**. 9. ed. São Paulo: Atlas, 2021.

MARTIN, Robert C. **Arquitetura limpa: o guia do artesão para estrutura e design de software**. Rio de Janeiro: Alta Books, 2020.

MICROSOFT. **ASP.NET Core Documentation (.NET 10)**. Redmond: Microsoft Corporation, 2026. Disponível em: https://learn.microsoft.com/pt-br/aspnet/core/. Acesso em: 16 out. 2026.

OSTERWALDER, Alexander; PIGNEUR, Yves. **Business Model Generation: inovação em modelos de negócios**. Rio de Janeiro: Alta Books, 2020.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software: uma abordagem profissional**. 9. ed. Porto Alegre: AMGH, 2021.

SCHWABER, Ken; SUTHERLAND, Jeff. **Guia do Scrum: as regras do jogo**. Boston: Scrum.org, 2020.

SEVERINO, Antônio Joaquim. **Metodologia do trabalho científico**. 24. ed. São Paulo: Cortez, 2017.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson Education do Brasil, 2019.

APÊNDICE A – CÓDIGO-FONTE DO PROJETO DESENVOLVIDO (WEB, API E MOBILE)

Repositório oficial PIM IV (Web .NET 10, APIs REST, Mobile Flutter e Scripts SQL): https://github.com/vinicius311006/projeto-pim-sistema-ong-main

Repositório base PIM III: https://github.com/VitorHAngelo/projeto-pim-sistema-ong

APÊNDICE B – VÍDEO DE AVALIAÇÃO "THINK ALOUD" DO SISTEMA

Disponível em: https://youtu.be/5aTXKtRjJZU

APÊNDICE C – MANUAL DO USUÁRIO DO SISTEMA

Disponível em: https://tinyurl.com/66uubhbs